CSCI 4061 - Lab8

11/02/2026



UNIVERSITY OF MINNESOTA
Driven to Discover®

Coverage

- Producer-Consumer Queues
- Condition Variables
- Thread-Safe Queue (queue.c)

Producer-Consumer Queues

Threads pass work through a shared, bounded queue. You'll build one again in Project 3.

`Shared head, size`	Every read or write of the list happens while holding the mutex
`Queue full`	A producer can't add yet. It sleeps until a consumer makes room
`Queue empty`	A consumer can't remove yet. It sleeps until a producer adds an item

```
# threads_main.c: 1 producer, 5 consumers, 10 messages, MAX_QUEUE_SIZE = 5
producer  --queue_add()⟶  [ msg | msg | msg |     |     ]  --queue_remove()⟶  consumers 0-4
```

Condition Variables

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t ready_cv = PTHREAD_COND_INITIALIZER;
int ready = 0;
void *waiter(void *arg) {
    pthread_mutex_lock(&lock);
    while (!ready)
        pthread_cond_wait(&ready_cv, &lock);
    pthread_mutex_unlock(&lock);
    return NULL;
}
void notify(void) {
    pthread_mutex_lock(&lock);
    ready = 1;
    pthread_cond_signal(&ready_cv);
    pthread_mutex_unlock(&lock);
}
```

`pthread_cond_wait(&cv, &lock)`

Unlocks and sleeps in one step, then relocks before returning. Call it while holding the lock.

`pthread_cond_signal(&cv)`

Wakes one thread waiting on cv. If nobody is waiting, the signal is simply lost.

`pthread_cond_broadcast(&cv)`

Wakes every waiting thread. They still take turns reacquiring the lock.

Init and errors

Use pthread_cond_init or PTHREAD_COND_INITIALIZER. Check return values.

Waiting in a Loop

```
int take(void) {
    pthread_mutex_lock(&lock);
    while (count == 0)  // nothing to take
        pthread_cond_wait(&not_empty, &lock);
    int item = buf[out];
    out = (out + 1) % N;
    count--;
    pthread_cond_signal(&not_full);  // room now
    pthread_mutex_unlock(&lock);
    return item;
}
```

Use while, not if. Wakeups can be spurious, and another consumer may grab the item first.

wait gives up the lock while asleep so other threads can change count, then takes it back.

After changing the state, signal the other side: a take makes room, an add makes an item.

Every early return must unlock first, including when wait or signal fails.

Lab 08: Make queue.c Thread-Safe

queue.c: queue_remove (starter)

```
1  // TODO synchronize before
2  strcpy(item_dest, queue->head->message);
3  node_t *new_head = queue->head->next;
4  free(queue->head);
5  queue->head = new_head;
6  queue->size--;
7  // TODO ...and after
```

queue.h (provided)

```
typedef struct {
    node_t *head;
    unsigned size;
    pthread_mutex_t lock;
    pthread_cond_t queue_full;
    pthread_cond_t queue_empty;
} queue_t;
```

queue_init

Initialize the mutex and both condition variables. Return -1 if any init fails.

queue_add

Wait on queue_full while the queue is full. Signal queue_empty after adding.

queue_remove

Wait on queue_empty while it's empty. Signal queue_full after removing.